

Projet de Recherche Opérationnelle

Modélisation et Résolution de Trois Problèmes d'Optimisation

Auteur du Projet

Année Universitaire 2025-2026

Table des matières

1	Introduction	2
2	Catégorie Logistique : Problème de Tournées de Véhicules (VRP)	2
2.1	Description	2
2.2	Modèle Mathématique	2
2.3	Méthode de Résolution	2
2.4	Code Python (Extrait clés)	2
2.5	Résultats et Interprétation	3
3	Catégorie Industrie : Ordonnancement de Machines	3
3.1	Description	3
3.2	Modèle Mathématique	3
3.3	Méthode de Résolution	3
3.4	Code Python (Extrait clés)	3
3.5	Résultats et Interprétation	4
4	Catégorie Mauritanie : Transport de Poissons (Nouadhibou - Nouakchott)	4
4.1	Description	4
4.2	Modèle Mathématique	4
4.3	Méthode de Résolution	4
4.4	Code Python (Extrait clés)	4
4.5	Résultats et Interprétation	5
5	Conclusion et Perspectives	5

1 Introduction

La Recherche Opérationnelle (RO) est une discipline essentielle pour la prise de décision analytique et l'optimisation des systèmes complexes. Dans le cadre de ce projet académique, nous avons sélectionné et résolu trois problèmes distincts afin de démontrer l'application de la RO dans divers domaines :

1. **Logistique** : Le problème des tournées de véhicules (VRP).
2. **Industrie** : Le problème d'ordonnancement de machines (Job Shop Scheduling).
3. **Problèmes réels en Mauritanie** : L'optimisation du transport de poissons entre Nouadhibou et Nouakchott.

Ces problèmes illustrent l'importance d'outils d'optimisation tels que Google OR-Tools et PuLP pour formuler et résoudre efficacement des contraintes opérationnelles, réduisant ainsi les coûts et maximisant l'efficacité.

2 Catégorie Logistique : Problème de Tournées de Véhicules (VRP)

2.1 Description

Le VRP (Vehicle Routing Problem) consiste à déterminer un ensemble optimal de routes pour une flotte de véhicules, devant desservir un ensemble de clients depuis un dépôt, en minimisant la distance totale parcourue et en respectant les capacités des camions.

2.2 Modèle Mathématique

- **Variables de décision** : $x_{ijk} \in \{0, 1\}$ valant 1 si le véhicule k se déplace du nœud i au nœud j , 0 sinon.
- **Fonction Objectif** : Minimiser la distance totale.

$$\text{Minimiser } Z = \sum_k \sum_i \sum_j c_{ij} x_{ijk}$$

où c_{ij} est la distance entre le nœud i et le nœud j .

- **Contraintes** :
 - Chaque client est visité exactement une fois.
 - La charge totale des clients sur une route ne dépasse pas la capacité du véhicule Q .
 - Continuité des flux (tout véhicule qui arrive à un client doit en repartir).

2.3 Méthode de Résolution

Nous avons modélisé ce problème avec la bibliothèque OR-Tools en Python, utilisant l'algorithme "Guided Local Search" (GLS) comme méta-heuristique pour échapper aux optima locaux et trouver une bonne solution en temps limité (1 seconde).

2.4 Code Python (Extrait clés)

```
1 # Définition des contraintes de capacité
2 def demand_callback(from_index):
3     from_node = manager.IndexToNode(from_index)
4     return data['demands'][from_node]
```

```

5
6 demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback
7 )
8 routing.AddDimensionWithVehicleCapacity(
9     demand_callback_index,
10     0, # null capacity slack
11     data['vehicle_capacities'], # maximum capacities
12     True, # start cumul to zero
13     'Capacity')

```

2.5 Résultats et Interprétation

L'algorithme a généré des itinéraires pour nos 3 véhicules, en s'assurant qu'aucun ne dépasse sa capacité de 30 unités tout en desservant l'ensemble des demandes. La distance totale minimisée obtenue offre une planification efficace pour une entreprise de livraison.

3 Catégorie Industrie : Ordonnancement de Machines

3.1 Description

Il s'agit d'un problème d'ordonnancement de type Job Shop. Différents "Jobs" (tâches globales) doivent être exécutés. Chaque job est composé d'une séquence de tâches, et chaque tâche doit être exécutée sur une machine spécifique avec une durée donnée. Le but est de minimiser le temps total d'exécution appelé Makespan.

3.2 Modèle Mathématique

- **Variables de décision** : $S_{ij} \geq 0$: la date de début de la tâche j du job i .
- **Fonction Objectif** : Minimiser le *Makespan* (la date de fin de la dernière tâche de l'ensemble des jobs).
- **Contraintes** :
 - **Précédence** : $S_{ij} + p_{ij} \leq S_{i,j+1}$ (une tâche d'un même job ne commence qu'après la fin de la précédente).
 - **Disjonction** : Deux tâches nécessitant la même machine ne peuvent se chevaucher dans le temps.

3.3 Méthode de Résolution

Nous utilisons le solveur CP-SAT (Constraint Programming) de Google OR-Tools. Il permet de gérer naturellement des variables de type "Intervalle" (`NewIntervalVar`) et des contraintes de non-chevauchement (`AddNoOverlap`).

3.4 Code Python (Extrait clés)

```

1 # Contraintes de non-chevauchement sur les machines
2 for machine in all_machines:
3     model.AddNoOverlap(machine_to_intervals[machine])
4
5 # Contraintes de precedence dans un job
6 for job_id, job in enumerate(jobs_data):
7     for task_id in range(len(job) - 1):
8         model.Add(all_tasks[job_id, task_id + 1].start >= all_tasks[job_id,
9 task_id].end)

```

3.5 Résultats et Interprétation

Le solveur trouve l'ordonnancement optimal très rapidement. En évitant les temps morts et les chevauchements de manière intelligente, le modèle trouve le makespan global optimal qui permet d'augmenter le rendement de l'usine manufacturière et de garantir le respect des délais.

4 Catégorie Mauritanie : Transport de Poissons (Nouadhibou - Nouakchott)

4.1 Description

Nouadhibou est le grand port de pêche de Mauritanie. Pour approvisionner la capitale, Nouakchott, nous devons transporter différents types de poissons (Courbine, Daurade, Mulet). Nous avons à disposition deux types de camions frigorifiques (10 Tonnes et 20 Tonnes) avec des coûts d'affrètement différents. Le but est de déterminer combien de camions de chaque type affréter pour satisfaire la demande au moindre coût.

4.2 Modèle Mathématique

Soit $C = \{10T, 20T\}$ l'ensemble des types de camions et $P = \{\text{Courbine}, \text{Daurade}, \text{Mulet}\}$ l'ensemble des espèces de poissons.

— **Variables de décision :**

- $x_p \geq 0$: La quantité de poisson $p \in P$ transportée (en Tonnes).
- $y_c \in \mathbb{N}$: Le nombre de camions de type $c \in C$ utilisés.

— **Fonction Objectif :** Minimiser le coût de transport.

$$\text{Minimiser } Z = \sum_{c \in C} \text{Cost}_c \times y_c$$

— **Contraintes :**

- Demande (satisfaire Nouakchott) : $x_p \geq \text{Demande}_p \quad \forall p \in P$
- Capacité maximale (Offre de Nouadhibou) : $x_p \leq \text{Offre}_p \quad \forall p \in P$
- Capacité totale des camions : $\sum_{p \in P} x_p \leq \sum_{c \in C} \text{Cap}_c \times y_c$

4.3 Méthode de Résolution

C'est un problème de Programmation Linéaire en Nombres Entiers (PLNE). Nous utilisons la bibliothèque PuLP en Python avec son solveur CBC par défaut pour trouver la répartition optimale des camions et minimiser les coûts logistiques.

4.4 Code Python (Extrait clés)

```
1 # Fonction Objectif : Minimiser le cout d'utilisation des camions
2 prob += pulp.lpSum([nb_camions[c] * cout_par_camion[c] for c in
   types_camions])
3
4 # Contrainte de capacite globale
5 prob += pulp.lpSum([nb_camions[c] * capacite_camions[c] for c in
   types_camions]) >= \
6     pulp.lpSum([quantites[p] for p in types_poissons])
```

4.5 Résultats et Interprétation

L'exécution indique précisément le nombre de camions 10T et 20T à affréter pour un coût de transport minimal. Le modèle garantit que la quantité totale de poissons respecte les contraintes d'offre à Nouadhibou et couvre exactement la demande à Nouakchott, empêchant ainsi le gaspillage de charge et rationalisant la flotte.

5 Conclusion et Perspectives

Ce projet a permis de comprendre comment formuler et implémenter des modèles de RO de manière concrète. Qu'il s'agisse de distribuer des biens (VRP), de planifier des processus industriels (Job Shop) ou de gérer la logistique de la pêche mauritanienne (Transport), l'optimisation aide à prendre des décisions rentables et justifiées mathématiquement. En perspective, ces modèles pourraient être améliorés : le problème VRP pourrait inclure des fenêtres de temps (VRPTW), l'ordonnancement intégrer des contraintes de ressources supplémentaires, et le transport de poissons tenir compte des coûts de stockage frigorifique sur place.

Références

- Documentation Google OR-Tools : <https://developers.google.com/optimization>
- Documentation de PuLP : <https://coin-or.github.io/pulp/>
- Support de cours de Recherche Opérationnelle, 2026.